VIBE-CODED APP

Security, Compliance & Reliability Master Prompt

A practical, implementation-focused prompt to audit, fix, and test a web app built with AI coding tools.

Coverage	Privacy, legal pages, authentication, API protection, payments, database performance, file uploads, resilience, monitoring, backups, and release validation.
Use with	Lovable, Bolt, Replit, Cursor, Claude Code, Google Antigravity, or another coding agent. Adapt to your actual stack and integrations.
Important	This is a technical implementation brief, not legal advice or a guarantee of compliance. Have qualified counsel review legal wording and confirm requirements for your markets.

How to use this document

- Copy the master prompt in Section 1 into your coding agent with the repository connected.
- Tell the agent to inspect the existing codebase first and work in small, reviewable phases—not to replace the app or rewrite the stack unnecessarily.
- Require evidence: code changes, migrations, automated tests, manual test steps, and a final list of anything still blocked or requiring a human decision.
- Do not paste live API keys, payment secrets, production customer data, or passwords into prompts or source code.

1. Copy-and-paste master prompt

Replace bracketed values where known. If a value is unknown, instruct the coding agent to inspect the repository and ask only when a decision is genuinely required.

```
ROLE
Act as a senior application security engineer, privacy engineer, backend engineer, QA lead, and
production reliability engineer. Audit and harden my existing vibe-coded application. Make changes
directly in the current repository where tools permit. Do not merely provide advice or a checklist:
inspect the code, identify real issues, implement safe fixes, and test them.

APP CONTEXT
- App name: [APP NAME]
- Production URL: [URL, if available]
- Frontend/framework: determine from repository
- Backend/API: determine from repository
- Database/auth provider: determine from repository
- Hosting/deployment: determine from repository
- Payment provider: [Stripe/Razorpay/other/not used]
- Target markets: [India/EEA/UK/US/other]
- Data handled: [account data, uploads, payments, health/financial data, etc.]
- Features not used: explicitly mark as not applicable; do not invent features.

NON-NEGOTIABLE WORKING RULES
1. Start by mapping the architecture, routes, data flows, auth model, database schema, storage,
third-party APIs, payment/subscription flows, environment variables, and deployment configuration. Read
existing project instructions first.
2. Produce a prioritized audit: Critical / High / Medium / Low, with evidence (file/path, function,
route, or configuration) and likely impact. Fix Critical and High issues before cosmetic work.
3. Preserve current product behavior and design unless a change is needed for security or correctness.
Do not rebuild from scratch or introduce a new framework without explaining why.
4. Never expose, print, commit, or copy real secrets. Do not place private API keys, database
credentials, signing secrets, service-role keys, or payment secret keys in frontend code, public
environment variables, browser bundles, logs, or error messages. If a secret may have been exposed, flag
it for rotation; do not claim rotation was completed unless it actually was.
5. Use server-side authorization for every protected resource and action. Hiding a button or relying on
frontend checks is not security.
6. Use least privilege, secure defaults, validated inputs, parameterized database operations, safe
output encoding, and deny-by-default access rules.
7. Before destructive schema changes or data migrations, explain impact and provide a reversible
migration or backup/rollback plan. Never delete production data as a test.
8. Use established libraries and provider-supported security mechanisms. Do not implement custom
cryptography.
9. Do not claim compliance or successful testing without evidence. If credentials, services, or
environment access are missing, state exactly what could not be verified and provide a safe manual
verification plan.
10. Make changes in small phases. After each phase, run relevant tests, lint, type checks, and build if
available. Report failures honestly.

WORKSTREAM A — LEGAL, PRIVACY, CONSENT & USER RIGHTS
A1. Terms and Conditions of Service: add a clearly linked Terms page describing service scope, user
responsibilities, account rules, prohibited use, subscriptions/payment terms where relevant,
intellectual property, suspension/termination, service availability, changes to terms, governing-law
placeholder for counsel review, and contact details. Do not invent a company address or legal entity.
Record effective/updated dates.
A2. Privacy Policy: document the actual data collected, purposes, legal bases where applicable,
processors/vendors, analytics/advertising, retention periods, international transfers, security
practices, user rights, children's data handling, contact route, and complaint/regulator route where
applicable. Derive statements from actual code/configuration; flag unknowns rather than fabricate them.
A3. Cookie consent: inventory cookies, local storage, SDKs, analytics and tracking. Categorize strictly
necessary vs optional. Where required by law, block non-essential cookies/trackers until valid consent.
Provide accept, reject, and granular settings with equally clear choices, consent withdrawal, and a
record of consent version/time/preferences. Do not describe local storage as a cookie if it is not one,
but include relevant tracking technologies in disclosures.
A4. GDPR/privacy compliance: assess applicability based on users, locations, processing and business
model. Implement data minimization, purpose limitation, retention, access/correction/export/deletion
workflows, consent records where applicable, processor inventory, and appropriate privacy-by-design
controls. Support lawful basis and data-subject request handling; do not claim that a banner alone makes
the app GDPR compliant.
A5. Age verification: determine whether the service has a real age restriction or children's privacy
obligations. If applicable, collect the minimum necessary age signal, avoid retaining identity
```

documents unless necessary, prevent access where legally required, and provide an appropriate guardian/age-handling process. Do not create a fake age gate that is trivially bypassable or collect unnecessary sensitive information.
A6. Clear disclaimers: add product-specific disclaimers for AI-generated outputs, health/financial/legal advice, user-generated content, availability, and third-party services only where relevant. Make them understandable and visible at the point of use; do not use disclaimers to excuse unsafe behavior.
A7. Deletion rights: implement account deletion and, where applicable, data export/access/correction. Explain what is deleted, anonymized, or retained for legitimate legal/security obligations and for how long. Require re-authentication for sensitive actions, revoke sessions/tokens, delete or anonymize related records and files, and handle cascades safely. Provide status and confirmation. Do not promise instant deletion if backups or legal retention mean otherwise.
A8. Copyright and license compliance: identify third-party assets, fonts, icons, libraries, templates, datasets and code. Check licenses and attribution requirements; produce a dependency/license inventory and flag incompatible, unknown, or missing licenses. Do not claim to perform a definitive copyright clearance or use unlicensed assets.
A9. License compliance: review open-source dependency licenses and any commercial API/service terms relevant to actual integrations. Pin or lock dependencies where appropriate, remove unused dependencies, and document required notices.
A10. Limit liability: draft only a counsel-review placeholder for a reasonable limitation-of-liability clause consistent with the service and applicable law. Do not insert unenforceable blanket language or imply legal review has occurred.
A11. Refund policy: reflect the real billing model and payment provider. State eligibility, time windows, cancellation vs refund distinction, trial/renewal rules, contact method and expected process. Ensure it does not conflict with consumer rights or payment-provider rules. Do not invent a refund window; use [BUSINESS DECISION REQUIRED] if unknown.
A12. Accessibility: target WCAG 2.2 AA where feasible and relevant. Audit semantic structure, keyboard navigation, focus visibility, labels, error messages, contrast, zoom/reflow, reduced motion, captions/alt text, and screen-reader announcements. Add automated checks plus manual test notes; automated scans alone do not prove conformance.

WORKSTREAM B — AUTHENTICATION & ACCOUNT SECURITY
B1. Test sign-up, login, logout, session expiry, email verification, password reset, and account recovery end-to-end. Verify both success and failure paths, expired/used links, invalid inputs, and cross-account isolation.
B2. Use the auth provider’s secure password hashing and session/token handling. Never store plaintext passwords or build custom password cryptography. Use secure, HttpOnly, SameSite cookies where the architecture supports them; use Secure in production and protect against CSRF where applicable.
B3. Require email verification for actions that depend on a verified address, according to product risk. Prevent verification tokens from being reused or accepted after expiry. Avoid revealing whether an email address has an account in sensitive flows.
B4. Verify password-reset tokens are random, short-lived, single-use, securely stored/validated, and invalidate relevant sessions when appropriate. Return generic responses that resist account enumeration.
B5. If OAuth (Google/Apple/etc.) exists, validate state/nonce and redirect URIs, use PKCE where appropriate, validate issuer/audience/signature through the provider/library, and link accounts safely. Do not enable OAuth if it is not part of the product.
B6. Apply rate limits and abuse controls to login, signup, password reset, verification resend, OTP, sensitive account changes, and expensive endpoints. Consider IP and account/device signals, progressive delays, and safe lockout design that avoids easy denial-of-service against a victim.
B7. Enforce authorization on the server for every user-owned record and admin action. Test horizontal and vertical privilege escalation, IDOR/BOLA, role manipulation, and tenant isolation. Never trust user-supplied user_id, role, price, plan, or ownership fields.
B8. Set appropriate session lifetimes, refresh-token rotation/revocation, logout invalidation, and re-authentication for sensitive changes. Remove tokens from URLs and logs. Add secure headers and CSRF protections as appropriate.

WORKSTREAM C — API, INPUT, UPLOAD & SECRET SECURITY
C1. Add server-side schema validation for all request bodies, query parameters, path parameters, headers used for trust decisions, and webhook payloads. Reject invalid types, unexpected fields where safe, oversized inputs, and malformed encodings.
C2. Add endpoint-specific rate limits, API quotas, request size/time limits, concurrency controls and per-user limits. Apply limits at the server/API gateway, not only in the UI. Ensure distributed deployments use a shared limiter or provider-native controls.
C3. Set spending caps and budget alerts for paid APIs/cloud services where supported. Use separate development/staging/production keys, least-privilege credentials, usage monitoring, and fail-closed behavior for exhausted budgets where feasible. Never rely on a frontend cap for cost control.
C4. Remove secret keys from frontend source, client environment variables, source maps, logs and repository history where possible. Frontend values are public even if environment-variable names look private. Route privileged API calls through a secured backend or serverless function. Rotate exposed secrets with the relevant provider; document manual rotation steps if no provider access is available.
C5. Add safe error handling: user-facing messages should be useful but generic; internal logs may

contain a correlation ID and sanitized diagnostic details. Never return stack traces, SQL errors, secrets, tokens, internal paths, or provider credentials to clients.
C6. Validate upload file size, extension, MIME type and file signature; use allowlists; generate server-side filenames; store outside executable/public paths or in private object storage; enforce authorization on downloads; scan for malware where appropriate; strip risky metadata if needed; and use short-lived signed URLs. Protect against path traversal, polyglot files, decompression bombs and content-type spoofing.
C7. Set CORS to explicit trusted origins, methods and headers. Do not use wildcard origins with credentials. Add security headers appropriate to the stack, including CSP, HSTS in production, frame protections, nosniff and a suitable referrer policy. Avoid breaking required integrations; test each header.
C8. Prevent injection and unsafe rendering: use parameterized queries/ORM safely, contextual output encoding, safe HTML sanitization if rich text is allowed, and safe URL handling. Review XSS, SQL/NoSQL injection, command injection, SSRF, path traversal, open redirects and insecure deserialization risks relevant to the codebase.
C9. Verify webhook signatures with provider-supported methods, reject replayed or stale events where applicable, and process events idempotently. Never trust a client redirect or frontend success screen as proof of payment.
C10. Keep dependencies patched. Run dependency, secret, static-analysis and vulnerability scans available in the project/toolchain. Review transitive dependencies and remove unused packages. Do not automatically upgrade major versions without checking breaking changes.

WORKSTREAM D — DATABASE, QUERY & FILE PERFORMANCE
D1. Review schema, query patterns and access policies. Add indexes only where justified by frequent filters, joins, sorting, uniqueness, or measured query plans. Consider write overhead and storage costs. Provide migrations and validate them on a staging/safe copy.
D2. Add server-side pagination to large result sets. Prefer cursor/keyset pagination for large or frequently changing datasets; enforce maximum page size and stable ordering. Do not fetch all rows and paginate only in the browser.
D3. Select only needed columns, avoid N+1 queries, batch related reads, and use query plans/metrics to identify slow queries. Apply sensible timeouts and avoid unbounded joins or full-table scans on hot paths.
D4. Compress suitable text responses and static assets using hosting/CDN capabilities. Optimize images, use responsive formats and caching, and avoid compressing already-compressed media unnecessarily.
D5. Cache repeat requests only when safe. Define cache keys, TTLs and invalidation rules; include tenant/user context where necessary; never serve private data across users. Avoid caching sensitive responses in shared caches unless deliberately secured.
D6. Limit upload and request sizes at both edge and application layers. Return clear 413-style errors where appropriate. Validate content before expensive processing.
D7. For large jobs, use background processing/queues when available, with bounded retries, idempotency, dead-letter handling and status reporting. Do not hold HTTP requests open for unnecessarily long tasks.

WORKSTREAM E — PAYMENTS & SUBSCRIPTIONS (ONLY IF THE APP USES THEM)
E1. Use the payment provider’s hosted checkout or approved SDK where practical; never collect/store raw card details unless the architecture is explicitly certified and designed for that. Keep PCI scope as low as possible.
E2. Verify payment webhooks server-side. Reconcile provider event IDs, payment IDs, order IDs, currency, amount and user/plan mapping. Treat payment as successful only after trusted provider confirmation.
E3. Prevent duplicate payments and duplicate subscription creation through idempotency keys, unique database constraints and transactional state transitions. Handle retries, double-clicks, browser refreshes, webhook retries and concurrent requests.
E4. Make payment/order creation atomic where possible. Use explicit states such as pending, succeeded, failed, cancelled, refunded and disputed. Never trust price, discount, tax or plan amount supplied by the client; calculate/validate on the server.
E5. Prevent duplicate subscriptions for the same intended entitlement. Make webhook handling idempotent and safe when events arrive out of order. Define cancellation, renewal, proration, failed renewal, refund and chargeback behavior.
E6. Display clear checkout totals, currency, renewal frequency and terms before confirmation. Ensure terms and refund policy match actual billing behavior and local consumer-law requirements.
E7. Test sandbox flows for success, decline, timeout, webhook delay, duplicate webhook, partial failure, cancellation, refund and reconciliation. Do not run real charges during tests.

WORKSTREAM F — RESILIENCE, ERRORS & UX STATES
F1. Add explicit loading, success, empty, partial-data, offline and error states to important screens. Prevent double submission and communicate retry options.
F2. Handle failed requests consistently: classify validation, authentication, authorization, rate-limit, timeout, provider and server errors. Retry only safe/idempotent operations with bounded exponential backoff and jitter. Do not automatically retry payment creation or other non-idempotent actions without idempotency protection.
F3. Set sensible connection/read/overall timeouts for external APIs. Use circuit breakers or graceful degradation for unreliable dependencies when appropriate. Do not allow a slow provider to exhaust all server workers.

F4. Cache repeat requests when appropriate and invalidate correctly after writes. Display stale data clearly if stale-while-revalidate is used. Avoid caching errors or private data improperly.
F5. Add structured error logging with request/correlation IDs, environment, route, error class and sanitized context. Redact passwords, tokens, session IDs, payment details, sensitive personal data and uploaded content. Apply access controls and retention to logs.
F6. Add uptime/health monitoring for critical services, including liveness/readiness checks where supported, alert routing and actionable thresholds. Health endpoints must not expose secrets or internal infrastructure details.
F7. Test simultaneous users and concurrent writes. Check race conditions in account creation, inventory/credits/usage limits, subscription state, file processing and payment flows. Use database transactions, unique constraints, atomic updates and idempotency as appropriate.
F8. Define graceful degradation when a third-party API is down or quota is exhausted. Preserve user work, show actionable status and avoid infinite spinners or silent failures.

WORKSTREAM G — BACKUPS, DEPLOYMENT & OPERATIONS
G1. Confirm database and file-storage backup configuration, retention, encryption, access control and backup frequency. Verify that backups are actually restorable in a non-production environment. A backup job succeeding is not proof of recoverability.
G2. Document recovery objectives (RPO/RTO) as business decisions if not specified. Test restoration, migrations and application compatibility. Do not restore over production during testing.
G3. Separate dev/staging/production environments and secrets. Verify production debug mode is off, admin routes are protected, test accounts/data are not exposed, and storage/database policies are production-safe.
G4. Add deployment checks: lint, type check, unit tests, integration tests, build, migration validation, dependency/secret scan and smoke tests as supported. Block release for Critical/High issues unless an authorized owner explicitly accepts documented risk.
G5. Add monitoring for latency, error rate, uptime, database saturation, queue backlog, external API failures, spending/quota usage, suspicious auth activity and payment reconciliation. Do not send sensitive data to monitoring vendors unnecessarily.
G6. Create an incident response note: how to disable a compromised key, rotate credentials, revoke sessions, disable a vulnerable route, preserve evidence, notify responsible owners and evaluate legal notification duties.

WORKSTREAM H — TEST PLAN & ACCEPTANCE CRITERIA
Create automated tests where practical and provide manual steps for gaps. Include:
- Auth: valid/invalid credentials, verification, expired reset links, logout, session expiry, OAuth state/redirect checks if used.
- Authorization: user A cannot read/change/delete user B’s records or files; non-admin cannot perform admin actions.
- Abuse: rate-limit thresholds, oversized requests, brute-force resistance, pagination limits and API quota enforcement.
- Input/upload: malformed JSON, unexpected fields, XSS/injection probes, wrong MIME/signature, oversized files and unauthorized downloads.
- Payments: duplicate clicks, replayed/duplicate webhooks, wrong amount/currency, timeouts, retries, failed payment, cancellation and refund in sandbox.
- Privacy: consent choices persist and can be changed; data export/deletion paths work; retained data is documented.
- Performance: representative large data set, database indexes/query plan, pagination, cache isolation and concurrent-user test.
- Reliability: external API timeout/failure, retry behavior, logging redaction, uptime alert test and backup restore drill.
- Accessibility: keyboard-only flow, visible focus, labels, error announcements, contrast and responsive zoom/reflow.
- Release: production build, migration safety, secrets scan and smoke tests.

DELIVERABLES
1. Architecture and data-flow summary.
2. Findings table with severity, evidence, impact, fix, and verification.
3. Implemented changes grouped by workstream.
4. Database migrations and rollback instructions, if any.
5. Tests added and exact commands/results; do not fabricate results.
6. Manual verification steps for anything not testable in the current environment.
7. Required provider/dashboard actions (key rotation, budget caps, webhook setup, backups, alerts) clearly separated from code changes.
8. A list of legal/business decisions requiring a human, including jurisdiction, age threshold, retention periods, refund windows, governing law, contact details and policy wording.
9. Remaining risks and prioritized next steps.

DEFINITION OF DONE
- No known Critical/High security issue remains unaddressed without a documented owner-approved exception.

```
- No privileged secret is present in client bundles or repository source.
- Authorization is enforced server-side and cross-user isolation tests pass.
- Rate limits, input validation, safe errors, upload restrictions and payment idempotency are verified
where applicable.
- Privacy/legal pages describe actual behavior and unknown legal decisions are clearly flagged.
- Build and tests pass, or failures are reported with evidence and next steps.
- Backups have a documented restore test or an explicit statement that restore testing remains pending.
- Provide a concise final report. Never state “fully secure,” “100% compliant,” or “production-ready”
without qualification and evidence.
```

2. Focused prompts for each checklist item

Use these as follow-up prompts if your coding agent needs to work one topic at a time. They are intentionally specific so the agent must inspect, implement, and verify instead of only describing best practices.

1. Terms and Conditions of Service

Inspect actual features, user flows, billing and jurisdiction assumptions. Create a Terms page linked in the footer and relevant signup/checkout flows. Use clear language and placeholders for legal decisions that are unknown. Do not invent legal entity, address, governing law, or consumer-rights terms. Add effective date and a review checklist for counsel.

2. Privacy Policy

Trace all personal data from collection through storage, analytics, AI providers, logging, sharing and deletion. Write a policy that reflects observed behavior, including purpose, retention, processors, transfers and rights. Flag any unknown retention/vendor details rather than guessing. Link it from signup and the footer.

3. Cookie consent

Inventory cookies, local storage, SDKs, pixels and analytics. Implement consent preferences with necessary/optional categories, clear accept and reject choices, granular controls where required, withdrawal and versioned records. Prevent optional trackers from loading before consent where legally required. Test all consent states.

4. GDPR and privacy compliance

Assess applicability; implement data minimization, purpose limitation, access/export/correction/deletion workflows, retention controls, processor inventory and consent records where relevant. Verify vendor DPAs and international-transfer settings as a human review item. Do not claim compliance solely because a policy or banner exists.

5. Age verification

Determine whether age checks are legally/product-required. If required, implement a proportionate method that minimizes collected data, blocks restricted access reliably, handles errors and documents retention. Do not collect IDs or biometrics unless necessary and legally reviewed. If not applicable, document why.

6. Secure payments

If payments exist, audit checkout, server-side pricing, provider webhook signatures, order state transitions, refunds, subscriptions, idempotency and reconciliation. Never trust frontend success, client-supplied price, or redirects as proof. Test sandbox success/failure/timeout/replay/duplicate events; never create real test charges.

7. Data encryption

Verify HTTPS/TLS in transit, provider-managed encryption at rest for databases and object storage, encrypted backups, and secret management. Use approved cryptographic libraries only. Avoid custom encryption. Identify sensitive fields and determine whether field-level encryption is warranted; document key access/rotation and limitations.

8. Accessibility standards

Audit against WCAG 2.2 AA as the target: semantic HTML, keyboard access, focus order/visibility, form labels, error announcements, contrast, zoom/reflow, alt text, captions and reduced motion. Fix issues and run automated scans plus manual keyboard/screen-reader checks. Report what remains unverified.

9. Copyright check

Inventory images, icons, fonts, code snippets, templates, datasets and user-provided content sources. Record source and license where known; replace assets with unknown or incompatible rights when appropriate. Do not claim definitive legal clearance; list items needing human confirmation.

10. Clear disclaimers

Review the app's actual outputs and risk areas. Add contextual, readable disclaimers for AI limitations, third-party data, health/financial/legal content, availability or user content only where relevant. Ensure the product does not rely on disclaimers instead of safety controls.

11. Data deletion rights

Implement a secure account/data deletion flow with confirmation and re-authentication for sensitive actions. Revoke sessions, delete/anonymize associated records and files, handle dependencies, and record status. Document legal-retention exceptions and backup expiry. Test deletion with a disposable account and verify that other users' data is untouched.

12. License compliance

Generate a dependency and license inventory from the lockfile and asset manifest. Identify unused packages, required notices/attributions, incompatible or unknown licenses and provider terms that require review. Preserve a reproducible lockfile and do not upgrade major versions without testing.

13. Limit liability

Draft a clearly marked counsel-review placeholder tailored to the real product, service model and jurisdictions. Do not use sweeping or misleading clauses or remove mandatory consumer rights. List the legal decisions needed before publication.

14. Refund policy

Inspect the payment provider and subscription behavior. Draft a policy covering eligibility, cancellation, renewal, trial conversion, refund process and contact channel based on real business rules. If the owner has not chosen a time window or exceptions, flag the decision instead of inventing it. Test policy consistency with checkout and billing.

15. Signup/login flow tested

Create and run end-to-end tests for signup, login, logout, duplicate signup, invalid credentials, disabled/unverified accounts, session expiry and cross-user isolation. Verify server-side authorization for every protected route. Report exact test commands and results.

16. Email verification

Verify email links are signed/random, expire, are single-use, and cannot verify another account. Rate-limit resend, use safe generic responses, handle expired links clearly, and test provider delivery configuration in staging. Never log full verification tokens.

17. Password reset

Test reset request, delivery, expired/used token, token reuse, account enumeration, password policy and session revocation. Store/validate reset tokens securely and make them single-use. Avoid revealing whether an email is registered.

18. OAuth (if included)

If OAuth is configured, validate state/nonce, redirect URI allowlists, issuer/audience/signature and PKCE where appropriate. Prevent unsafe account linking and open redirects. Test cancelled, failed, replayed and mismatched callback flows. If OAuth is absent, do not add it just for this task.

19. Rate limiting / brute-force prevention

Add server-side endpoint-specific limits for authentication, OTP, password reset, sensitive changes, uploads and expensive APIs. Consider IP and account-level controls, safe progressive delays, shared storage for multiple instances, monitoring and clear 429 responses. Test boundary behavior and avoid easy victim-account lockout.

20. Database indexes

Inspect query patterns and database query plans. Recommend and add indexes for frequent filters, joins, sorting and uniqueness only when justified. Include safe migrations, rollback notes and validation of write overhead. Never add indexes blindly or run risky production migrations without a plan.

21. Paginate large results

Find endpoints that return unbounded records. Add server-side pagination with a maximum page size, stable ordering, appropriate indexes and cursor/keyset pagination where suitable. Validate input and test large datasets, changing records, empty pages and unauthorized data access.

22. Compress files and responses

Use hosting/CDN or server middleware to compress text responses and optimize static assets/images. Avoid recompressing already-compressed media. Verify content types, cache headers, performance impact and that compression does not introduce security regressions for sensitive reflected responses.

23. Limit upload size

Enforce upload/request size limits at edge and application layers. Validate extension, MIME and file signature; generate filenames; store privately; authorize downloads; use signed URLs; and scan risky files where appropriate. Test oversized, spoofed, malformed and unauthorized upload/download cases.

24. API limits and quotas

Set per-endpoint, per-user and global quotas for expensive operations and external API calls. Enforce server-side, define burst behavior, return useful 429 errors and monitor consumption. Protect provider keys and ensure one user cannot exhaust shared quotas.

25. Spending caps

Identify paid cloud and AI/API dependencies. Configure provider-side budgets, alerts, quotas and key separation where available; implement internal usage accounting and hard stops for costly operations when feasible. State which settings require dashboard access. Do not claim a budget cap exists unless verified in the provider.

26. Error handling

Centralize error handling with safe client messages, stable error codes and correlation IDs. Map validation/auth/rate-limit/timeout/provider/internal errors correctly. Redact secrets and personal data; never expose stack traces or raw database/provider errors in production.

27. Loading states

Audit every asynchronous user action and add accessible loading/progress states, disabled duplicate-submit protection, timeout feedback and recovery options. Ensure controls do not become permanently stuck after failures. Test slow networks and repeated clicks.

28. Empty states

Create useful empty states for first-use, no search results, no history and no filtered results. Include a relevant next action without exposing private data or suggesting features that do not exist. Ensure empty state is distinct from loading and error states.

29. Cache repeated requests

Identify repeated safe reads and add caching only with a clear key, TTL, invalidation and failure strategy. Include tenant/user identity where necessary; do not leak private data between users. Test stale data, post-write invalidation and cache-disabled behavior.

30. Failed requests and retries

Handle network failures, 4xx/5xx responses and provider errors consistently. Retry only transient, safe/idempotent operations with bounded exponential backoff and jitter. Do not retry payment/order creation without idempotency. Preserve user input and display a recoverable message.

31. Uptime monitoring

Configure health checks and external uptime monitoring for critical user journeys and dependencies. Define actionable thresholds, notification recipients and escalation. Keep health endpoints minimal and secret-free. If provider dashboard access is missing, provide exact setup steps and mark configuration pending.

32. Error logging

Implement structured logs and error reporting with correlation IDs, environment, route and sanitized diagnostic context. Redact authorization headers, passwords, tokens, payment data, personal data and file contents. Define access controls, retention and alert rules; test redaction explicitly.

33. Prevent duplicate subscriptions

Use provider subscription IDs, unique constraints and idempotent server-side operations to prevent duplicate subscriptions. Handle retries, repeated clicks, webhook replays and out-of-order events. Make state transitions transactional and reconcile local records with provider state.

34. Prevent duplicate payments

Use provider-supported idempotency keys and unique order/payment records. Validate amount/currency server-side, verify webhooks and make payment state transitions atomic. Test concurrent clicks, timeout/retry, duplicate webhook and browser refresh in sandbox.

35. API timeouts

Set connection, read and total deadlines for external API calls. Bound concurrency, cancel abandoned work where possible and handle timeout distinctly. Use safe retries and circuit breakers only where suitable. Test slow/hung dependencies and verify requests do not exhaust workers.

36. Test simultaneous users

Run concurrency tests for account creation, credits/usage, records, inventory, subscription changes and payments where applicable. Identify race conditions and fix with transactions, atomic updates, unique constraints and idempotency. Use a staging environment and synthetic data; do not load-test production without authorization.

37. Backup and restore testing

Verify database and file backups, encryption, retention, access controls and schedule. Restore into a separate non-production environment, verify row/file counts and app functionality, and document recovery steps and measured duration. Do not overwrite production. If credentials are unavailable, list exact dashboard steps and mark unverified.

38. Frontend secret-key fix

Scan source, build output, public environment variables, source maps and repository history for privileged keys. Move privileged calls to server-side functions, use least-privilege secrets and rotate any exposed keys with the provider. Remove keys from client bundles and verify with a production build/secret scan. Assume any key previously shipped to browsers is compromised.

3. Release checklist

Do not treat this as a one-time checklist. Re-run relevant checks after major changes, new integrations, new data types, or billing changes.

- ■ Critical and High findings fixed or formally accepted by an accountable owner.
- ■ Privileged secrets absent from frontend bundles, public variables, logs and committed source; exposed keys rotated.
- ■ Server-side authorization tested across users, roles and tenants.
- ■ Authentication, verification and reset flows tested, including expired/replayed tokens.
- ■ Rate limits, API quotas, upload limits, timeouts and safe error handling verified.
- ■ Database indexes justified by query patterns; pagination enforced server-side.
- ■ Payment webhooks verified and payment/subscription creation is idempotent (if applicable).
- ■ Privacy, cookie, terms, refund, deletion and disclaimer pages reflect actual app behavior and have human legal review where needed.
- ■ Accessibility tested with automated tools and manual keyboard/screen-reader checks.
- ■ Logs redact secrets and sensitive data; monitoring and alerts have an owner.
- ■ Backup restore test completed in non-production, or explicitly marked pending.
- ■ CI/build/tests/migrations and production smoke tests pass, or failures are documented.
- ■ Rollback and incident/key-rotation steps documented.
- ■ Remaining risks, provider dashboard tasks, and business/legal decisions listed clearly.

4. Required final report format

```
## Executive summary
- Overall status: [Not ready / Ready with documented risks / Ready for review]
- Critical findings: [number]
- High findings: [number]
- Main risks: [short list]

## Changes implemented
- [Area] — [files/routes/config changed and what changed]

## Verification evidence
- Tests/commands run: [exact commands]
- Results: [pass/fail and concise evidence]
- Manual checks completed: [list]

## Actions requiring owner/provider access
- [Key rotation, dashboard budgets, webhook configuration, backups, monitoring, etc.]

## Legal/business decisions requiring human review
- [Jurisdiction, age requirements, retention, refund windows, contact details, policy clauses]

## Remaining risks and next steps
- [Severity, reason, recommended owner/action]
```

5. Practical notes

• **Legal compliance is contextual.** GDPR, cookie rules, age restrictions, accessibility obligations, refunds and liability rules depend on where you operate, who your users are, and what your app does. Get qualified legal review before relying on these pages.

• **Security cannot be proven by a prompt alone.** Require code inspection, tests, configuration review, provider-side settings, and ongoing monitoring.

• **Do not expose production data.** Use staging accounts and synthetic records for testing. Restrict access to logs and backups.

• **Vibe-coded apps need ongoing maintenance.** Re-run dependency/security checks and review permissions when adding AI APIs, OAuth, payments, analytics, uploads or admin tools.